

DL_and_classical_ML

July 13, 2026

Index 1-. Standalone LinearSVC model 2-. Validation Strategy for LinearSVC Model using K-Fold 3-. Standalone Convolutional Neural Network 4-. Validation Strategy for CNN Model using K-Fold 5-. Explainable AI for LinearSVC Model 6-. Explainable AI for CNN Model

```
[1]: # For LinearSVC
import os # for OS ops
import cv2 as opencv # Image editing
from sklearn.preprocessing import RobustScaler
from sklearn.svm import LinearSVC #Principial of LinearSVC
from sklearn import metrics
import numpy as np
import shap
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
import matplotlib.pyplot as plt

# For CNN:
import torch # main, provides the tensioners
from torchvision import transforms # Resize, normalize, crop, horizontally,
    ↪ flip, and convert images to tensors.
from torchvision import datasets # To list and sort the files to be analyzed
from torch.utils.data import DataLoader, Subset # Package the list of files to
    ↪ be processed.
import torch.nn.functional as functional
from collections import Counter
import torch.nn as nn # neural network building blocks
from torch.optim import SGD # backpropagation optimization algorithms
from sklearn.metrics import classification_report
from datetime import datetime
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    ↪ f1_score
```

Standalone LinearSVC model

```
[2]: pathTrain = "/media/gnirut/2TB/interlinked/data_set_pumonia/train"
pathTest = "/media/gnirut/2TB/interlinked/data_set_pumonia/test"
pathValidation = "/media/gnirut/2TB/interlinked/data_set_pumonia/val"
```

```
pathTrainAndTest = "/media/gnirut/2TB/interlinked/data_set_pumonia/  
↳train_and_test/"
```

```
[3]: # Custom process for converting images to arrays, given that Scikit does not  
↳have a default process for this  
def FromIMGtoArray(pathValidation_):  
    listaImagenes = []  
    listaEtiquetas = []  
    for eachclass in os.listdir(pathValidation_):  
        for imagen in os.listdir(f"{pathValidation_}/{eachclass}"):   
            imgArray = opencv.imread(f"{pathValidation_}/{eachclass}/{imagen}",  
↳opencv.IMREAD_GRAYSCALE)  
            imgArray = opencv.resize(imgArray, (256, 256), interpolation=opencv.  
↳INTER_LANCZOS4)  
            imgArray = imgArray / 255  
  
            label = None  
            if eachclass == "NORMAL":  
                label = 0  
            elif eachclass == "PNEUMONIA":  
                label = 1  
  
            listaImagenes.append(imgArray)  
            if len(imgArray.shape) > 2:  
                print(f"{pathValidation_}/{eachclass}/{imagen}")  
  
            listaEtiquetas.append(label)  
  
    X = np.array(listaImagenes)  
    Y = np.array(listaEtiquetas)  
  
    return X, Y
```

```
[4]: # applying the created function  
X_train, Y_train = FromIMGtoArray(pathTrain)  
X_test, Y_test = FromIMGtoArray(pathTest)  
X_vali, Y_vali = FromIMGtoArray(pathValidation)
```

```
[5]: # Reshaping the array  
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1] * X_train.shape[2])  
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1] * X_test.shape[2])  
X_vali = X_vali.reshape(X_vali.shape[0], X_vali.shape[1] * X_vali.shape[2])
```

```
[6]: #scaler = StandardScaler() # Not used, was not a good fit  
# To smooth outliers  
scaler = RobustScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_vali = scaler.transform(X_vali)
```

```
[7]: # When training the model itself, it is important to balance the weights as the
      ↪ labels are not balanced
      model = LinearSVC(max_iter=10000, C=5, random_state=42, class_weight="balanced")
      model.fit(X_train, Y_train)
```

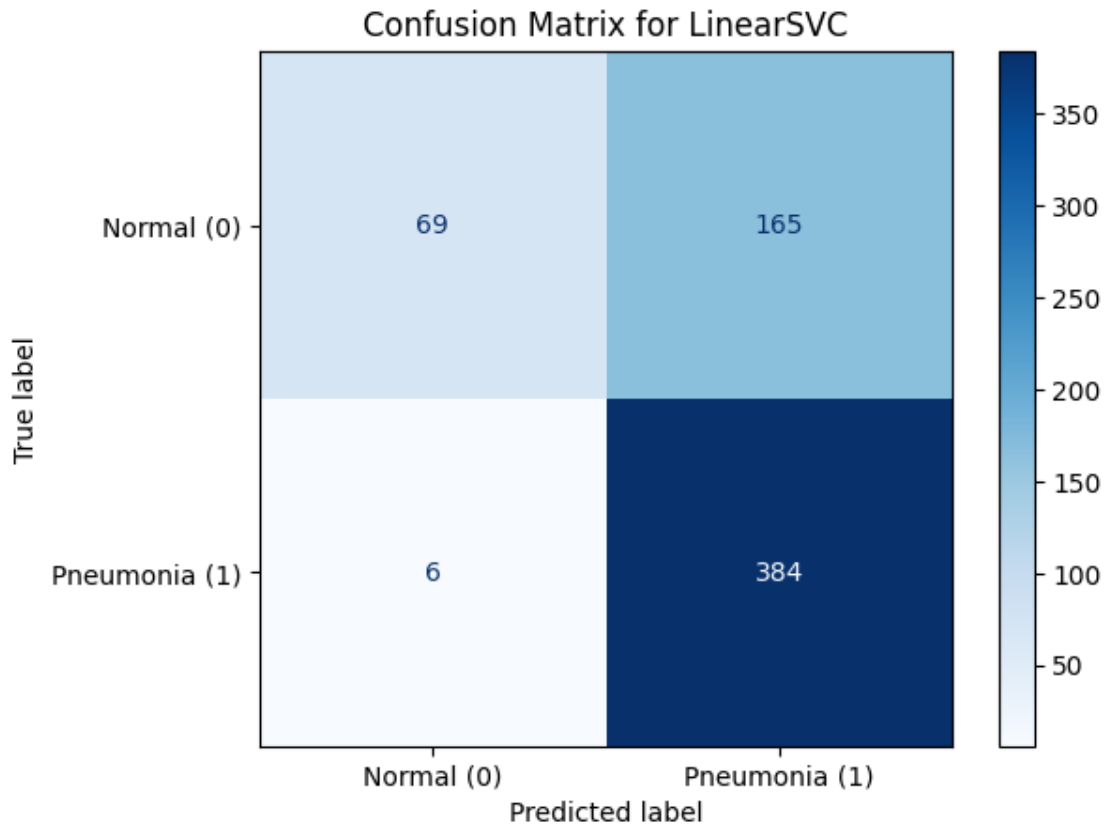
```
[7]: LinearSVC(C=5, class_weight='balanced', max_iter=10000, random_state=42)
```

```
[8]: # Testing for predictions
      Y_predicted = model.predict(X_test)
```

```
[9]: # detailed performance metrics based on the prediction
      print(metrics.classification_report(Y_test, Y_predicted,
      ↪ target_names=["NORMAL", "PNEUMONIA"]))
```

	precision	recall	f1-score	support
NORMAL	0.92	0.29	0.45	234
PNEUMONIA	0.70	0.98	0.82	390
accuracy			0.73	624
macro avg	0.81	0.64	0.63	624
weighted avg	0.78	0.73	0.68	624

```
[10]: metrics.ConfusionMatrixDisplay.from_predictions(Y_test, Y_predicted,
      ↪ display_labels=["Normal (0)", "Pneumonia (1)"], cmap="Blues")
      plt.title("Confusion Matrix for LinearSVC")
      plt.show()
```



Validation Strategy for LinearSVC Model using K-Fold

```
[11]: # Recycling the same function
X_merged, Y_merged = FromIMGtoArray(pathTrainAndTest)
X_merged = X_merged.reshape(X_merged.shape[0], X_merged.shape[1] * X_merged.
    ↪shape[2])

[12]: scaler = RobustScaler()
pipeline = Pipeline([("scaler", RobustScaler()), ("svc",
    ↪LinearSVC(max_iter=10000, C=5, random_state=42))])

[13]: # Model evaluation using stratified folds cross-validation with multiple metrics
evaluations = ["accuracy", "precision_macro", "recall_macro", "f1_macro"]
k_fold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = cross_validate(pipeline, X_merged, Y_merged, cv=k_fold,
    ↪scoring=evaluations)

[14]: # Checking the mean and standard deviation of each metric obtained in the
    ↪cross-validation
```

```

for metric in evaluations:
    scores = results[f"test_{metric}"]
    print(f"{metric}: {scores.mean():.4f} ± {scores.std():.4f}")

```

```

accuracy: 0.9384 ± 0.0073
precision_macro: 0.9222 ± 0.0094
recall_macro: 0.9214 ± 0.0108
f1_macro: 0.9217 ± 0.0093

```

Standalone Convolutional Neural Network

```

[15]: # Custom .txt log
timestamp = datetime.today().strftime("%Y-%m-%d %H:%M:%S")
with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
    w.write(f"{timestamp}\n")

[16]: # Transformations and preprocessing for training with data augmentation
transform_train = transforms.Compose([transforms.
    ↳ Grayscale(num_output_channels=3), # PyTorch works with 3 channels; we adjust
    ↳ for Batch Normalization.
                                     transforms.Resize(size=(256,256)), #
    ↳ capped to 256
                                     transforms.ColorJitter(brightness=0.2,
    ↳ contrast=0.2),
                                     transforms.RandomHorizontalFlip(p=0.5),
                                     transforms.ToTensor(),
                                     transforms.Normalize([0.485, 0.456, 0.
    ↳ 406], [0.229, 0.224, 0.225]))

transform_test = transforms.Compose([transforms.
    ↳ Grayscale(num_output_channels=3),
                                     transforms.Resize(size=(256,256)),
                                     transforms.ToTensor(),
                                     transforms.Normalize([0.485, 0.456, 0.
    ↳ 406], [0.229, 0.224, 0.225]))

[17]: # Load the image 3 datasets and displays the classes and distributions
trainset = datasets.ImageFolder(pathTrain, transform=transform_train)
testset = datasets.ImageFolder(pathTest, transform=transform_test)
valiset = datasets.ImageFolder(pathValidation, transform=transform_test)

print(trainset.classes)
print(Counter(trainset.targets))

```

```

['NORMAL', 'PNEUMONIA']
Counter({1: 3875, 0: 1341})

```

```
[18]: train_loader = DataLoader(trainset, batch_size=32, shuffle=True,
    ↳ num_workers=10, pin_memory=True, persistent_workers=True)
test_loader = DataLoader(testset, batch_size=32, shuffle=True, num_workers=10,
    ↳ pin_memory=True, persistent_workers=True)
vali_loader = DataLoader(valiset, batch_size=32, shuffle=True, num_workers=10,
    ↳ pin_memory=True, persistent_workers=True)
```

```
[19]: class RedNeuronal(nn.Module):
    def __init__(self):
        super().__init__() # calls the initialization method of the parent class

        # 3 = RGB channels, 32 = Output images created, 5 = 5x5 filter matrix
        # WITHOUT padding = ((input_size - kernel_size) / stride + 1)
        # WITH padding = (((input_size - kernel_size) + (2 * padding)) / stride
    ↳ + 1)
        # (Pixels on one side - Kernel) / Stride + 1 = Y; (out channels x (Y x
    ↳ Y))
        # Input parameters are mandatory, defined by the previous step; output
    ↳ parameters are optional and a matter of personal choice.

        output_channel_1 = 32 # activation maps AKA new images
        output_channel_2 = 64
        output_channel_3 = 128
        kernel_conv = 3
        kernel_pool = 2
        channels_rgb = 3
        final_pixels4_size = 32

        flatOutput1 = 128
        flatOutput2 = 64
        flatOutput3 = 2 # "NORMAL", "PNEUMONIA"

        self.conv1 = nn.Conv2d(channels_rgb, output_channel_1, kernel_conv,
    ↳ padding=1) # (((256 - 3) + (2 * 1)) / 1) + 1 = 252, output: 32 x 256 x 256
        self.pool1 = nn.MaxPool2d(kernel_pool, kernel_pool) # (256 - 2) / 2 + 1 =
    ↳ 128 output: 32 x 128 x 128
        self.conv2 = nn.Conv2d(output_channel_1, output_channel_2, kernel_conv,
    ↳ padding=1) # (((128 - 3) + (2 * 1)) / 1) + 1 = 128, output: 32 x 128 x 128
        self.pool2 = nn.MaxPool2d(kernel_pool, kernel_pool) # (128 - 2) / 2 + 1 =
    ↳ 61 output: 64 x 64 x 64
        self.conv3 = nn.Conv2d(output_channel_2, output_channel_3, kernel_conv,
    ↳ padding=1) # (((64 - 3) + (2 * 1)) / 1) + 1 = 64, output: 128 x 64 x 64
        self.conv4 = nn.Conv2d(output_channel_3, output_channel_3, kernel_conv,
    ↳ padding=1) # (((64 - 3) + (2 * 1)) / 1) + 1 = 64, output: 128 x 64 x 64
        self.conv5 = nn.Conv2d(output_channel_3, output_channel_3, kernel_conv,
    ↳ padding=1) # (((64 - 3) + (2 * 1)) / 1) + 1 = 64, output: 128 x 64 x 64
```

```

        self.pool3 = nn.MaxPool2d(kernel_pool, kernel_pool) # (64 - 2)/2 + 1 = 32
        ↪32 output: 128 x 32 x 32
        self.flat1 = nn.Flatten() # from vector to linear
        self.Dens1 = nn.Linear(output_channel_3 * final_pixels4_size *
        ↪final_pixels4_size, flatOutput1) # nn.Linear(in_features, out_features,
        ↪bias=True)
        self.Dens2 = nn.Linear(flatOutput1, flatOutput2)
        self.drop = nn.Dropout(p=0.2)
        self.Dens3 = nn.Linear(flatOutput2, flatOutput3)

    def forward(self, x):
        x = self.pool1(functional.relu(self.conv1(x)))
        x = self.pool2(functional.relu(self.conv2(x)))
        x = functional.relu(self.conv3(x))
        x = functional.relu(self.conv4(x))
        x = functional.relu(self.conv5(x))
        x = self.pool3(x)
        x = self.flat1(x)
        x = functional.relu(self.Dens1(x))
        x = functional.relu(self.Dens2(x))
        x = self.drop(x)
        x = self.Dens3(x)
        return x

```

```

[20]: print(torch.cuda.get_device_name(0))
device = torch.device("cuda")

```

NVIDIA GeForce RTX 3060 Ti

```

[21]: net = RedNeuronal()
net.to(device) # To be run in the GPU
total = torch.tensor([1341, 3875], dtype=torch.float32) # Create an array and
        ↪convert it to tensors.
pesos = total.sum() / (2.0 * total)
pesos = pesos.to(device)

lossFunction = nn.CrossEntropyLoss(weight=pesos) # Softmax included,
optimizer = SGD(net.parameters(), lr=0.001, momentum=0.9) # Stochastic Gradient
        ↪Descent

```

```

[22]: rounds = 50

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=rounds / 2,
        ↪gamma=0.5) # useful for making the learning rate progressive

precisiones = np.array([])
maximo = 0

```

```

for epoch in range(rounds):
    runningLoss = 0.00

    for i, data in enumerate(train_loader):
        inputs, labels = data
        inputs, labels = inputs.to(device, non_blocking=True), labels.
        to(device, non_blocking=True)

        optimizer.zero_grad() # Clear the stored derivatives
        outputs = net(inputs) # Forward pass: produces the 2 logits per image
        loss = lossFunction(outputs, labels) # Versus de outputs vs labels
        loss.backward() # calculates derivatives and stores them inside the
        model parameters
        optimizer.step() # Apply the gradients to update model weights

        runningLoss += loss.item()

    scheduler.step() # Adjust the learning rate.

    # Pre-training inference
    correct = 0
    total = 0

    net.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for data in test_loader:
            images, labels = data
            images, labels = images.to(device, non_blocking=True), labels.
            to(device, non_blocking=True)
            outputs = net(images)
            _, predicted = torch.max(outputs, 1)
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    status_report = classification_report(all_labels, all_preds,
        target_names=["Normal", "Pneumonia"], output_dict=True)
    accuracy = round(status_report["accuracy"] * 100, 2)
    precisiones = np.append(precisiones, accuracy)
    promedio = round(precisiones.mean(), 2)

    if promedio > maximo:
        maximo = promedio

```



```

with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
    w.write(f"[{epoch}] Updated loss: {runningLoss / len(train_loader)} @_
↳Last Accuracy: {accuracy} Average: {promedio} Maximo: {maximo}\n")

# weight saved
torch.save(net.state_dict(), "/media/gnirut/2TB/interlinked/modelo_entrenado.
↳pth")

timestamp = datetime.today().strftime("%Y-%m-%d %H:%M:%S")
with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
    w.write(f"{timestamp}\n\n")

```

```

/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

```

[23]: # Load
net = RedNeuronal()
net.to(device) # To run un GPU, otherwise slow CPU
net.load_state_dict(torch.load("/media/gnirut/2TB/interlinked/modelo_entrenado.
↳pth"))

# Post-training inference
correct = 0
total = 0

net.eval()

all_predictions = []
all_labels = []

with torch.no_grad():
    for data in test_loader:
        images, labels = data

```

```

        images, labels = images.to(device, non_blocking=True), labels.
        ↪to(device, non_blocking=True)
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        all_predictions.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

```

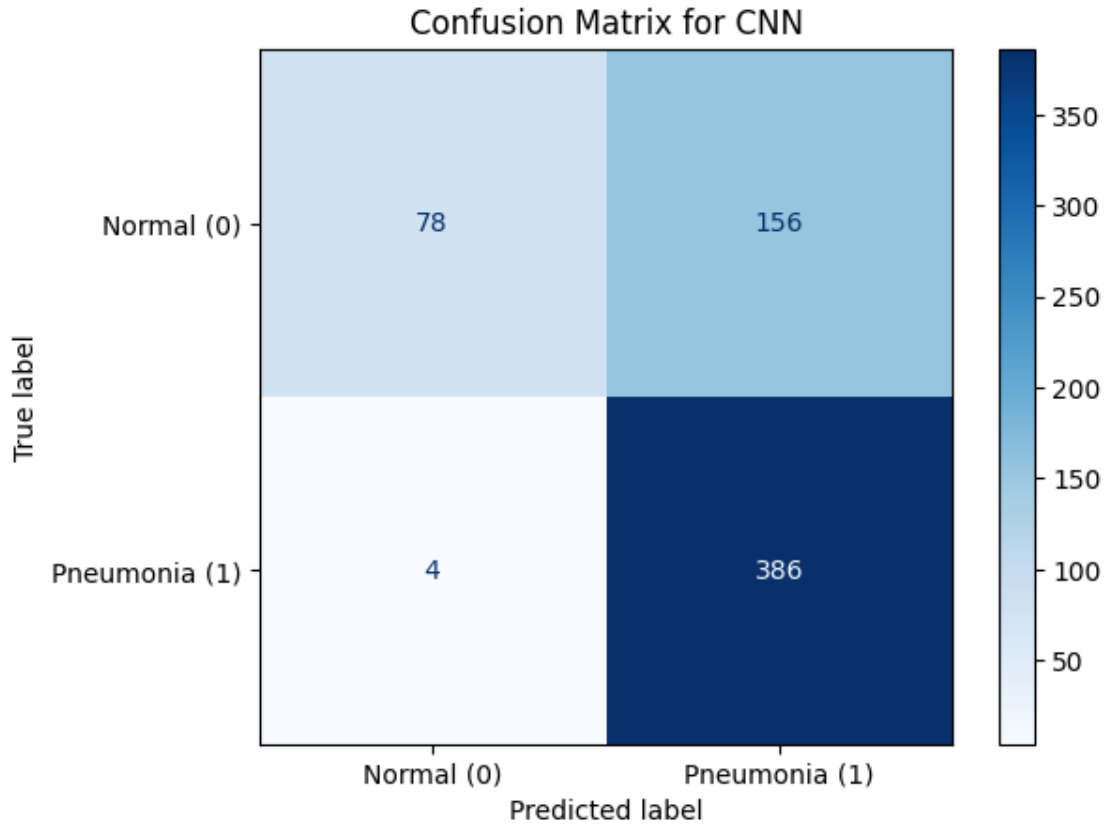
```

[24]: # Displays the classification report and confusion matrix for the CNN model.
print(classification_report(all_labels, all_predictions,
    ↪target_names=["Normal", "Pneumonia"]))

metrics.ConfusionMatrixDisplay.from_predictions(all_labels, all_predictions,
    ↪display_labels=["Normal (0)", "Pneumonia (1)"], cmap="Blues")
plt.title("Confusion Matrix for CNN")
plt.show()

```

	precision	recall	f1-score	support
Normal	0.95	0.33	0.49	234
Pneumonia	0.71	0.99	0.83	390
accuracy			0.74	624
macro avg	0.83	0.66	0.66	624
weighted avg	0.80	0.74	0.70	624



Validation Strategy for CNN Model using K-Fold

```
[25]: pathTrainAndTest = "/media/gnirut/2TB/interlinked/data_set_pumonia/
      ↪train_and_test"

[26]: # Stratified cross-validation setup and metrics
k_fold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
evaluations = ["accuracy", "precision_macro", "recall_macro", "f1_macro"]

[27]: trainset = datasets.ImageFolder(pathTrainAndTest, transform=transform_train)
testset = datasets.ImageFolder(pathTrainAndTest, transform=transform_test)

[28]: timestamp = datetime.today().strftime("%Y-%m-%d %H:%M:%S")
with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
    w.write(f"{timestamp}\n")

fold_results = {"accuracy": [], "precision": [], "recall": [], "f1": []}

# Iterate over each fold obtaining the train and test indices for
↪cross-validation
```

```

for fold, (train_index, test_index) in enumerate(k_fold.split(np.
    ↪zeros(len(trainset)), np.array(trainset.targets))):
    train_loader = Subset(trainset, train_index)
    test_loader = Subset(testset, test_index)

    train_loader = DataLoader(train_loader, batch_size=32, shuffle=True,
    ↪num_workers=10, pin_memory=True, persistent_workers=True)
    test_loader = DataLoader(test_loader, batch_size=32, shuffle=True,
    ↪num_workers=10, pin_memory=True, persistent_workers=True)

    net = RedNeuronal()
    net.to(device) # To be run in the GPU
    lossFunction = nn.CrossEntropyLoss() # Softmax included,
    optimizer = SGD(net.parameters(), lr=0.001, momentum=0.9) # Stochastic
    ↪Gradient Descent

    rounds = 50

    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=rounds /
    ↪2, gamma=0.5)

    precisiones = np.array([])
    maximo = 0

    for epoch in range(rounds):
        runningLoss = 0.00

        for i, data in enumerate(train_loader):
            inputs, labels = data
            inputs, labels = inputs.to(device, non_blocking=True), labels.
            ↪to(device, non_blocking=True)

            optimizer.zero_grad() # Clear the stored derivatives
            outputs = net(inputs) # Forward pass
            loss = lossFunction(outputs, labels) # Versus de outputs vs labels
            loss.backward() # calculates derivatives and stores them inside the
            ↪model parameters
            optimizer.step() # Apply the gradients to update model weights

            runningLoss += loss.item()

        scheduler.step() # learning rate

    correct = 0
    total = 0

```

```

net.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for data in test_loader:
        images, labels = data
        images, labels = images.to(device, non_blocking=True), labels.
        to(device, non_blocking=True)
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    status_report = classification_report(all_labels, all_preds,
        target_names=["Normal", "Pneumonia"], output_dict=True)
    accuracy = round(status_report["accuracy"] * 100, 2)
    precisiones = np.append(precisiones, accuracy)
    promedio = round(precisiones.mean(), 2)

    if promedio > maximo:
        maximo = promedio

    with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
        w.write(f"[{epoch}] Updated loss: {runningLoss / len(train_loader)}\n")
        @ Last Accuracy: {accuracy} Average: {promedio} Maximo: {maximo}\n")

    torch.save(net.state_dict(), f"/media/gnirut/2TB/interlinked/
        modelo_entrenado{fold}.pth")

    timestamp = datetime.today().strftime("%Y-%m-%d %H:%M:%S")
    with open("/media/gnirut/2TB/interlinked/log_pytorch.txt", "a+") as w:
        w.write(f"{timestamp}\n\n")

    fold_results["accuracy"].append(accuracy_score(all_labels, all_preds))
    fold_results["precision"].append(precision_score(all_labels, all_preds,
        average="macro"))
    fold_results["recall"].append(recall_score(all_labels, all_preds,
        average="macro"))
    fold_results["f1"].append(f1_score(all_labels, all_preds, average="macro"))

```

/home/gnirut/.local/lib/python3.10/site-packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.

```

Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use ``zero_division`` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
```

```
warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```



```

samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/gnirut/.local/lib/python3.10/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

```
[29]: fold_results
```

```

[29]: {'accuracy': [0.9606164383561644,
0.9417808219178082,
0.9657534246575342,
0.9563356164383562,
0.9580479452054794],
'precision': [0.9570797941739053,
0.919395672869197,
0.9618687880348729,
0.9538948302433101,
0.9441103916127043],
'recall': [0.9420011537244832,
0.9371145722845606,
0.950523828132269,
0.9340646457879752,
0.950254005470887],
'f1': [0.9491841544753279,
0.9276627369832272,
0.9559960969140756,
0.9433625895299518,
0.9471182143342238]}

```

Explainable AI for LinearSVC Model

```

[30]: # Train a LinearSVC model and calculate SHAP values to interpret feature
↳importance
model = LinearSVC(max_iter=100000, C=5, random_state=42,
↳class_weight="balanced")

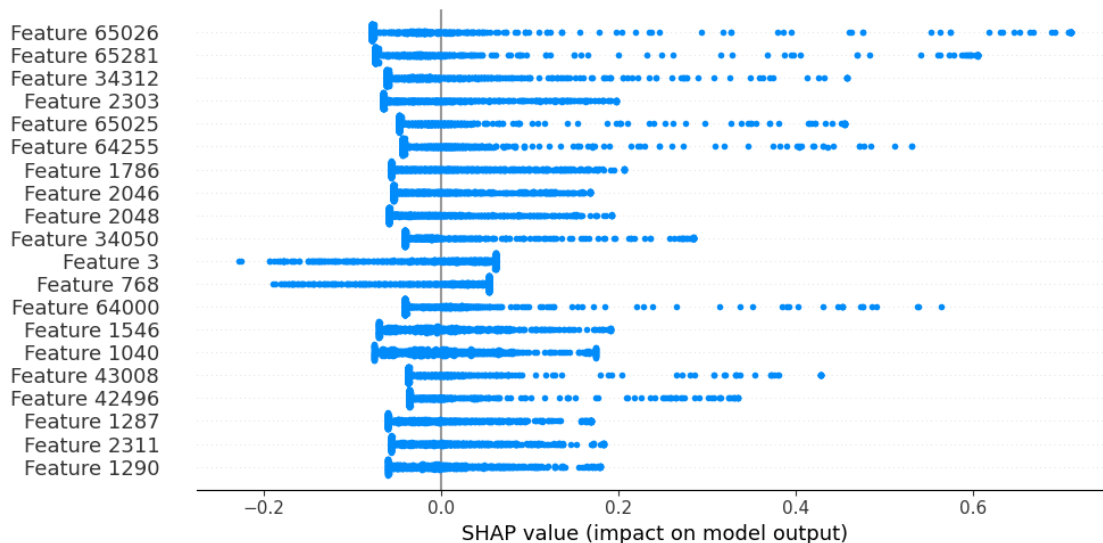
```

```

model.fit(X_merged, Y_merged)

# SHAP for LinearSVC (Executed but not useful)
sample = shap.sample(X_test, 100)
explainer = shap.LinearExplainer(model, sample)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, plot_size=(10, 5))

```



[30]:

```

[31]: # Train a LinearSVC and generate pixel contribution heatmaps for each image
model = LinearSVC(max_iter=10000, C=5, random_state=42, class_weight="balanced")
model.fit(X_train, Y_train)

for imageIndex in range(len(X_test)):
    x_ejemplo = X_test[imageIndex]
    radiografia_ejemplo = x_ejemplo.reshape(256, 256)
    contribucion = (model.coef_[0] * x_ejemplo).reshape(256, 256)
    vmax = np.percentile(np.abs(contribucion), 99)
    fig, ax = plt.subplots()
    ax.imshow(radiografia_ejemplo, cmap="gray")
    im = ax.imshow(contribucion, cmap="bwr", alpha=0.5, vmin=-vmax, vmax=vmax)
    plt.colorbar(im, label="Contribution to prediction")
    plt.title(f"Heatmap. Image id: {imageIndex}, prediction: {model.
    ↪predict([x_ejemplo])[0]}, real: {int(Y_test[imageIndex])}")
    plt.savefig(f"/media/gnirut/2TB/interlinked/data_set_pumonia/
    ↪output_images_linearsvc/{imageIndex}")
    plt.close(fig)

```

```
[32]: # Generate predictions with the model and display the classification report
predicciones = model.predict(X_test)
print(metrics.classification_report(Y_test, predicciones,
    ↳target_names=["NORMAL", "PNEUMONIA"]))
```

	precision	recall	f1-score	support
NORMAL	0.92	0.29	0.45	234
PNEUMONIA	0.70	0.98	0.82	390
accuracy			0.73	624
macro avg	0.81	0.64	0.63	624
weighted avg	0.78	0.73	0.68	624

Explainable AI for CNN

```
[33]: # Class that implements Grad-CAM to generate heatmaps for the image influence
    ↳the models prediction
# Highly inspired by: https://medium.com/@okanyenigun/
    ↳what-is-grad-cam-visualising-cnn-predictions-step-by-step-9637a29a48fb
class GradCAMModel:
    def __init__(self, model_, target_layer):
        self.model = model_
        self.target_layer = target_layer
        self.activations = None
        self.gradients = None

        target_layer.register_forward_hook(self._save_activation)
        target_layer.register_full_backward_hook(self._save_gradient)

    def _save_activation(self, module, input, output):
        self.activations = output.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()

    def generate(self, input_tensor, class_idx=None):
        self.model.eval()
        output = self.model(input_tensor) # forward normal, dispara el hook

        if class_idx is None:
            class_idx = output.argmax(dim=1).item()

        self.model.zero_grad()
        loss = output[0, class_idx]
        loss.backward() # triggers the backward hook
```

```

    # Channel weighting
    pooled_gradients = torch.mean(self.gradients, dim=[0, 2, 3])
    activations = self.activations[0]

    for i in range(activations.shape[0]):
        activations[i, :, :] *= pooled_gradients[i]

    heatmap = torch.mean(activations, dim=0).cpu().numpy()
    heatmap = np.maximum(heatmap, 0) # ReLU
    heatmap /= (np.max(heatmap) + 1e-8) # normalizes to 0-1

    return heatmap, class_idx

```

```

[34]: # Load the trained model and apply Grad-CAM to each image in the test set ans_
      ↪ saving the generated heatmaps as files
net = RedNeuronal()
net.to(device)
net.load_state_dict(torch.load("/media/gnirut/2TB/interlinked/modelo_entrenado4.
      ↪ pth"))

gradcam = GradCAMModel(net, net.conv5)

for folio, (imagen, etiqueta) in enumerate(testset):
    input_tensor = imagen.unsqueeze(0).to(device)

    heatmap, clase_predicha = gradcam.generate(input_tensor)
    heatmap_resized = opencv.resize(heatmap, (256, 256))

    img_np = imagen.permute(1, 2, 0).cpu().numpy()
    img_np = (img_np - img_np.min()) / (img_np.max() - img_np.min())

    fig, ax = plt.subplots()
    ax.imshow(img_np[:, :, 0], cmap="gray")
    im = ax.imshow(heatmap_resized, cmap="jet", alpha=0.5)
    plt.colorbar(im, label="Grad-CAM activation")
    plt.title(f"Prediction: {testset.classes[clase_predicha]}, Real: {testset.
      ↪ classes[etiqueta]}")
    plt.savefig(f"/media/gnirut/2TB/interlinked/data_set_pumonia/
      ↪ output_images_cnn/{folio}")
    plt.close(fig)

```

Bibliography: Andreoni, R. (2023) ‘Support Vector Machine with Scikit-Learn: A Friendly Introduction’, Towards Data Science. Towards Data Science, 11 October. Available at: <https://towardsdatascience.com/support-vector-machine-with-scikit-learn-a-friendly-introduction-a2969f2ff00d/>.

Bir, P. (2019) ‘Image Classification with a Linear Classifier’, Medium. Medium, 13 Au-

gust. Available at: <https://medium.com/@paarthbir/image-classification-with-a-linear-classifier-cab02f7f8a30>.

Bishop, C.M. and Bishop, H. (2023) *Deep Learning: Foundations and Concepts*. Cham: Springer Cham. Available at: <https://doi.org/10.1007/978-3-031-45468-4>.

Jung, A. (2022) *Machine Learning: The Basics*. Singapore: Springer Singapore (Machine Learning: Foundations, Methodologies, and Applications). Available at: <https://doi.org/10.1007/978-981-16-8193-6>.

He, K. et al. (2015) ‘Deep Residual Learning for Image Recognition’, arXiv preprint [Preprint]. Available at: <https://doi.org/10.48550/arXiv.1512.03385>.

Kim, D.-K. (2025) ‘Grad-CAM: A Gradient-based Approach to Explainability in Deep Learning’, Medium. Medium, 17 February. Available at: <https://medium.com/@kdk199604/grad-cam-a-gradient-based-approach-to-explainability-in-deep-learning-871b3ab8a6ce>.

Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) ‘ImageNet Classification with Deep Convolutional Neural Networks’. *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, Curran Associates, Inc., pp. 1097–1105. Available at: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>.

Mishra, D. (2019) ‘Demystifying Convolutional Neural Networks using Grad-Cam’, Towards Data Science. Towards Data Science, 15 October. Available at: <https://towardsdatascience.com/demystifying-convolutional-neural-networks-using-gradcam-554a85dd4e48/>.

Mooney, P. (2018) ‘Chest X-Ray Images (Pneumonia)’. Kaggle. Available at: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>.

Mureverwi, B. (2022) ‘Churn Prediction with Artificial Neural Networks’, Python in Plain English. Python in Plain English (Medium), 12 February. Available at: <https://python.plainenglish.io/churn-prediction-with-artificial-neural-networks-73ae4179f5dd>.

Plaue, M. (2023) *Data Science: An Introduction to Statistics and Machine Learning*. Berlin, Heidelberg: Springer Berlin, Heidelberg. Available at: <https://doi.org/10.1007/978-3-662-67882-4>.

Santos, G. (2022) ‘A Basic Introduction to Pipelines in Scikit Learn’, Towards Data Science. Towards Data Science, 29 September. Available at: <https://towardsdatascience.com/a-basic-introduction-to-pipelines-in-scikit-learn-bd4cee34ad95/>.

Strbac, M. (2021) ‘pneumonia-detection-in-chest-xrays’. GitHub. Available at: <https://github.com/mariostrbac/pneumonia-detection-in-chest-xrays>.

Trevisan, V. (2022) ‘Using SHAP Values to Explain How Your Machine Learning Model Works’, Towards Data Science. Towards Data Science, 17 January. Available at: <https://towardsdatascience.com/using-shap-values-to-explain-how-your-machine-learning-model-works-732b3f40e137/>.